



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Aprendizaje automático - Sesión 4

Feature Selection & K-nearest Neighbors & K-means Algorithms

Doctorado en Inteligencia Artificial

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

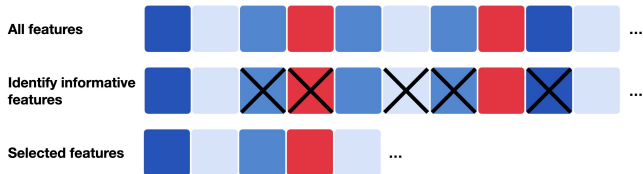
Tabla de Contenidos

- 1 Introducción
- 2 Conceptos básicos
- 3 K-means
- 4 Hierarchical Clustering
- 5 K-Nearest Neighbors
- 6 Técnicas de evaluación de algoritmos de clasificación

Introducción

Feature Selection

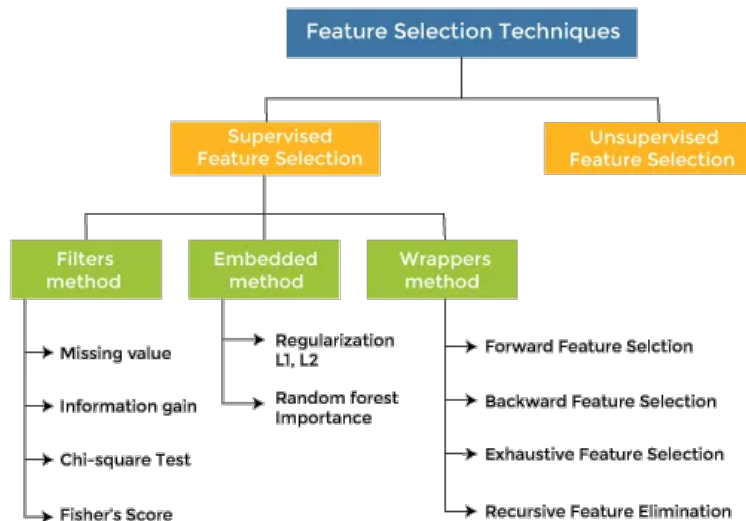
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p +$$



Dado un conjunto de datos con muchas variables de entrada, no es recomendable incluir todas las variables de entrada en la ecuación de regresión lineal.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p +$$

- ***Stepwise regression*** es una técnica de selección de características en la regresión lineal múltiple (MLR).
- Existen tres tipos de *stepwise regression*: **backward elimination, forward selection, and bidirectional elimination.**



Feature Selection

Su propósito principal es identificar y utilizar el subconjunto más relevante de características del dataset, descartando aquellas que son irrelevantes o redundantes - evitar sobreajuste.

- **Sequential Forward Selection:** Agrega características de manera secuencial sin reconsiderar las eliminaciones. Es más simple y directo.
- **Sequential Backward Selection:** Elimina características de manera secuencial sin reconsiderar las eliminaciones. Es más simple y directo.
- **Recursive Feature Elimination:** Elimina iterativamente la característica menos importante según un criterio específico, como el valor absoluto más bajo de los coeficientes en un modelo de regresión.
- **Forward Step-wise Selection (FSS):** Agrega características pero también puede eliminar características previamente agregadas si ya no mejoran el modelo. Es más dinámico y flexible.

Su propósito principal es identificar y utilizar el subconjunto más relevante de características del dataset, descartando aquellas que son irrelevantes o redundantes - evitar sobreajuste.

1. Técnicas secuenciales simples:

- **Forward Selection:** empieza con ninguna variable y las agrega una a una.
- **Backward Selection:** empieza con todas las variables y elimina las menos útiles.

2. Técnicas avanzadas:

- **Stepwise Selection:** agrega y elimina variables según su efecto en el modelo.
- **Recursive Feature Elimination (RFE):** elimina de forma iterativa la variable menos relevante según el modelo.

Backward Selection

Backward Selection es una técnica de selección de variables en regresión lineal. Comienza con todas las variables del conjunto de datos e itera eliminando una a una las que aportan menos al modelo, típicamente según su p-valor.

Pasos del algoritmo:

1. Ajustar un modelo con **todas las variables disponibles**.
2. Evaluar la **significancia estadística** de cada variable (**usualmente con p-valor o t-score**).
3. Eliminar la variable **menos significativa**.
4. Reajustar el modelo y repetir el proceso hasta que todas las variables restantes sean significativas.

Ventajas / Limitaciones:

- Reduce el riesgo de **sobreaajuste** al eliminar variables irrelevantes.
- Es simple de implementar y comprender.
- **Puede eliminar variables importantes si hay multicolinealidad¹.**
- No garantiza encontrar el mejor subconjunto global de variables.

¹ La multicolinealidad provoca que las varianzas de los estimadores de los coeficientes aumenten. Eso significa que los errores estándar son más grandes, lo que puede llevar a que variables que sí son relevantes muestren un p-value elevado

Forward Step-wise Selection (FSS)

Agrega variables de forma secuencial al modelo, comenzando con ninguna.

■ Paso 0: Modelo Vacío

- Iniciar con un modelo sin predictores (solo el intercepto).
- Evaluar el rendimiento inicial (ej., $R^2 \approx 0$).

■ Paso 1: Añadir la Primera Característica

- Evaluar modelos simples con cada característica individualmente.
- Seleccionar la característica que maximiza la mejora del rendimiento.
- Añadir permanentemente esta característica al modelo.

■ Paso 2: Añadir la Siguiente Característica

- Mantener las características seleccionadas previamente.
- Evaluar modelos añadiendo cada una de las características restantes.
- Seleccionar la característica restante que maximiza la mejora del rendimiento.
- Añadir permanentemente esta característica al modelo.

Forward Step-wise Selection (FSS)

Pasos Siguientes (iterar)

- Repetir el Paso 2.
- En cada paso, evaluar la adición de cada característica no seleccionada al conjunto actual.
- Añadir la característica que produce la mayor mejora en el rendimiento.

Criterios de Parada

- No hay mejora significativa en el rendimiento.
- Se alcanza un número predefinido de características.
- Criterios basados en pruebas estadísticas (ej., valor p del coeficiente).

Ventajas:

- Computacionalmente más eficiente que probar todas las combinaciones.
- Permite identificar subconjuntos relevantes de variables.

Ejemplo de Algoritmo FSS

```
1  def fss(x, y, names_x, k = 10000):
2      p = x.shape[1]-1
3      k = min(p, k)
4      names_x = np.array(names_x)
5      remaining = range(0, p)
6      selected = [p]
7      current_score = best_new_score = 0.0
8      while remaining and len(selected)<=k :
9          score_candidates = []
10         for candidate in remaining:
11             model = LinearRegression(fit_intercept=True, n_jobs=1)
12             indexes = selected + [candidate]
13             x_train = x[:,indexes]
14             predictions_train = model.fit(x_train, y).predict(x_train)
15             residuals_train = predictions_train - y
16             error_candidate = np.mean(np.power(residuals_train, 2))
17             score_candidates.append((error_candidate, candidate))
18         score_candidates.sort()
19         score_candidates[:] = score_candidates[::-1]
20         best_new_score, best_candidate = score_candidates.pop()
21         remaining.remove(best_candidate)
22         selected.append(best_candidate)
23         print "selected = %s ..." %names_x[best_candidate]
24         print "totalvars=%d, mse = %f" %(len(indexes),best_new_score)
25     return selected
26 features_fss = fss(Xstd_train, reg_label_train, names_regressors)
27 need_feat = features_fss[:5]
```

Cálculo del Z-score en Regresión Lineal

1. Ajuste del Modelo de Regresión:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p +$$

Ajustar el modelo para obtener los coeficientes $\hat{\beta}$ del modelo.

2. Obtener las Predicciones del Modelo:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p$$

Calcula las predicciones $\hat{y}$ utilizando el modelo ajustado.

3. Calcular la Varianza Residual ($\hat{\sigma}^2$):

$$\hat{\sigma}^2 = \frac{1}{N - p - 1} \sum_{m=1}^M (y_m - \hat{y}_m)^2,$$

donde N es el número de observaciones y p es el número de variables independientes.

4. Calcular la Matriz $(X^T X)^{-1}$:

$$\text{Var}(\hat{\beta}) = \hat{\sigma}^2 \cdot (X^T X)^{-1}$$

Se obtiene a partir de la matriz de X (variables independientes) y la ($\hat{\sigma}^2$).

4. Calcular la Matriz $(X^T X)^{-1}$:

$$\text{Var}(\hat{\beta}) = \hat{\sigma}^2 \cdot (X^T X)^{-1}$$

Se obtiene a partir de la matriz de X (variables independientes) y la $(\hat{\sigma}^2)$. Se selecciona la diagonal, por esto no se dice que es la $\text{Cov}(\beta_{ls}) = \sigma^2 (X^T X)^{-1}$ ([Sesión 2](#))

5. Calcular los Errores Estándar:

$$SE(\hat{\beta}_i) = \sqrt{\text{Var}(\hat{\beta}_i)}$$

Toma los elementos diagonales de la matriz de covarianza para obtener los errores estándar de cada coeficiente.

La varianza de $\hat{\beta}_i$ es el valor en la diagonal correspondiente de la matriz de covarianza.

6. Calcular el Z-score (o t-score):

$$Z_i = \frac{\hat{\beta}_i}{SE(\hat{\beta}_i)}$$

El Z-score mide cuántas desviaciones estándar está el coeficiente de cero.

Implementación en Python - Z-score

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn import datasets, linear_model
4 import pandas as pd
5 from scipy import stats
6
7 lm = LinearRegression()
8 lm.fit(X,y)
9 params = np.append(lm.intercept_,lm.coef_)
10 predictions = lm.predict(X)
11
12
13 newX = np.append(np.ones((len(X),1)), X, axis=1)
14 MSE = (sum((y-predictions)**2))/(len(newX)-len(newX[0]))
15
16 var_b = MSE*(np.linalg.inv(np.dot(newX.T,newX)).diagonal())
17 sd_b = np.sqrt(var_b)
18 ts_b = params/ sd_b
19 sd_b = np.round(sd_b,3)
20 ts_b = np.round(ts_b,3)
21 print('Z_score (t-value)', ts_b)
```

Z-score

1. age: Edad en años.
2. sex: Sexo (masculino o femenino).
3. bmi: Índice de masa corporal (BMI).
4. bp: Presión arterial media.
5. s1: Colesterol sérico total (TC).
6. s2: Lipoproteínas de baja densidad (LDL).
7. s3: Lipoproteínas de alta densidad (HDL).
8. s4: Relación colesterol total/HDL.
9. s5: Logaritmo del nivel de triglicéridos en suero (LTG).
10. s6: Nivel de glucosa en sangre (GLU).

	coef	std err	t
const	152.1335	2.576	59.061
x1	-10.0099	59.749	-0.168
x2	-239.8156	61.222	-3.917
x3	519.8459	66.533	7.813
x4	324.3846	65.422	4.958
x5	-792.1756	416.680	-1.901
x6	476.7390	339.030	1.406
x7	101.0433	212.531	0.475
x8	177.0632	161.476	1.097
x9	751.2737	171.900	4.370
x10	67.6267	65.984	1.025

- t-value grande (en valor absoluto): Los coeficientes son estadísticamente significativos.
- t-value pequeño (cercano a 0): Los coeficientes no son significativos y, por lo tanto, esa variable no tiene un efecto significativo en la variable dependiente.

Calcular el *p-value*

1. Calcular el estadístico t :
$$t = \frac{\hat{\beta}_i}{SE(\hat{\beta}_i)}$$

2. Usar la distribución t de Student con $n - p - 1$ grados de libertad:

$$p\text{-value} = 2 \cdot P(T > |t|), \quad T \sim t(df)$$

3. Interpretación:

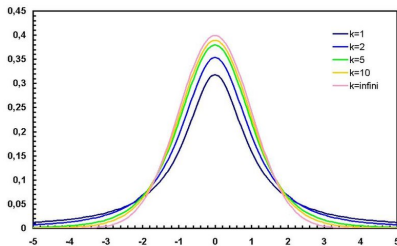
- p -valor pequeño ($< 0,05$): evidencia contra la hipótesis nula $H_0 : \beta_i = 0$.
- p -valor grande: no hay evidencia suficiente para rechazar H_0 .

Ejemplo: Supongamos una regresión con $n = 53$ observaciones y $p = 2$ variables explicativas. Para un coeficiente:

$$\begin{aligned}\hat{\beta}_i &= 0,5, & SE(\hat{\beta}_i) &= 0,2 \\ t &= \frac{0,5}{0,2} = 2,5, & df &= 53 - 2 - 1 = 50 \\ p\text{-value} &= 2 \cdot P(T_{50} > 2,5) \approx 0,016\end{aligned}$$

Como $p \approx 0,016 < 0,05$, la variable es significativa al 5 %.

Distribución t-Student



- Distribución de probabilidad que surge del problema de estimar la media de una población normalmente distribuida cuando el tamaño de la muestra es pequeño y la desviación estándar poblacional es desconocida.

Ejemplo: calcular p-value en python

```
1 from scipy import stats
2
3 # t-score observado y grados de libertad
4 t = 2.5
5 df = 50
6
7 # p-valor bilateral
8 p_value = 2 * (1 - stats.t.cdf(abs(t), df))
9 print(f"p-value: {p_value:.4f}") # Resultado: 0.0160
```

Consultar la distribución t

Se busca conocer cuán probable sería obtener un estadístico t tan extremo como el observado si la hipótesis nula fuera cierta (es decir, si $\beta_i = 0$).

Para eso, comparamos nuestro valor de t con la distribución t de Student, que depende de los grados de libertad del modelo:

$$df = n - p - 1$$

El área bajo la curva desde $|t|$ hacia los extremos nos da el **p-value**. Si ese área es pequeña, el valor observado es poco probable bajo H_0 , y lo rechazamos.

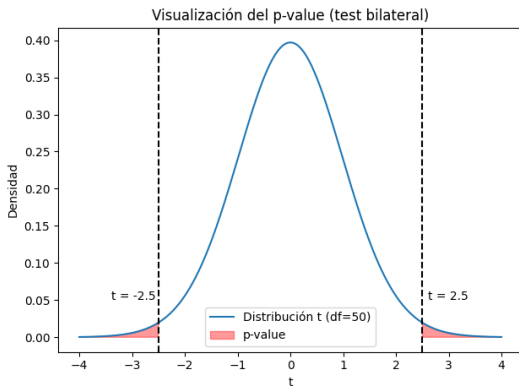
Visualizando el t-value y el p-value

Interpretación gráfica:

- El **t-value** es una posición en la distribución t de Student.
- El **p-value** es el área bajo la curva en ambas colas desde ese valor y su opuesto.
- Un **p pequeño** indica que el valor observado sería muy raro si $\beta = 0$.

Relación clave:

$$p = 2 \cdot \mathbb{P}(T > |t|)$$



Áreas sombreadas = p-value para $t = \pm 2,5$

Linear regression using Statsmodels library

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.datasets import fetch_california_housing
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 import statsmodels.api as sm
7
8 data = fetch_california_housing()
9 X, y = data.data, data.target
10 feature_names = data.feature_names
11
12 scaler = StandardScaler()
13 X_scaled = scaler.fit_transform(X)
14
15 X_df = pd.DataFrame(X_scaled, columns=feature_names)
16
17 X_train, X_test, y_train, y_test = train_test_split(X_df, y, test_size=0.3, random_state=42)
18
19 X_train_const = sm.add_constant(X_train)
20
21 model = sm.OLS(y_train, X_train_const).fit()
22
23 print(model.summary())
24
25 X_test_const = sm.add_constant(X_test)
26 predictions = model.predict(X_test_const)
27
28 r2_score = model.rsquared
29 print("\nR en conjunto de prueba:", round(r2_score, 4))
```

Linear regression using Statsmodels library

OLS Regression Results

```
=====
Dep. Variable:          y    R-squared:                0.609
Model:                  OLS    Adj. R-squared:           0.609
Method:                 Least Squares    F-statistic:        2815.
Date:                   Wed, 16 Apr 2025    Prob (F-statistic):    0.00
Time:                   07:56:16    Log-Likelihood:       -15823.
No. Observations:       14448    AIC:                  3.166e+04
Df Residuals:           14439    BIC:                  3.173e+04
Df Model:                8
Covariance Type:        nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
const          2.0689      0.006    343.624      0.000        2.057        2.081
MedInc          0.8470      0.009     89.283      0.000        0.828        0.866
HouseAge        0.1218      0.007     18.240      0.000        0.109        0.135
AveRooms       -0.3021      0.017    -17.362      0.000       -0.336       -0.268
AveBedrms       0.3690      0.017     21.978      0.000        0.336        0.402
Population     -0.0009      0.006     -0.138      0.890       -0.013        0.012
AveOccup       -0.0350      0.005     -6.889      0.000       -0.045       -0.025
Latitude       -0.0894      0.018    -49.265      0.000       -0.930       -0.858
Longitude      -0.0689      0.018    -48.757      0.000       -0.904       -0.834
=====
```

```
Omnibus:                2973.049    Durbin-Watson:                1.964
```

...

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpretación de los resultados: t y $P > |t|$

¿Qué representa el t -value?

Es el estadístico de prueba que indica cuántas desviaciones estándar está el coeficiente estimado $\hat{\beta}_i$ del valor nulo (0):

$$t_i = \frac{\hat{\beta}_i}{SE(\hat{\beta}_i)}$$

- Valores grandes (en valor absoluto) $\rightarrow$ mayor evidencia de que $\hat{\beta}_i \neq 0$.

¿Qué representa $P > |t|$?

Es el **p-valor** asociado al t-score. Indica la probabilidad de observar ese valor (o uno más extremo) si la hipótesis nula es cierta:

$$H_0 : \beta_i = 0$$

- Si $p < 0,05$: evidencia estadística suficiente para rechazar H_0 la variable es significativa.
- Si $p > 0,05$: no hay evidencia suficiente variable posiblemente irrelevante.

Los modelos de regresión lineal requieren variables numéricas. Las variables categóricas (como colores, géneros, regiones) deben ser transformadas para que el modelo las entienda. Las dummy variables permiten incorporar estas categorías de forma cuantitativa sin imponer un orden falso².

- Variables categóricas convertidas en valores 0 o 1. **Codificación one-hot**.
- Permiten que los modelos de regresión lineal trabajen con variables cualitativas.

Ejemplo:

Color	Dummy_Rojo	Dummy_Verde	Dummy_Azul
Rojo	1	0	0
Verde	0	1	0
Azul	0	0	1

Modelo de Regresión:

$$y = \beta_0 + \beta_1(\text{Dummy_Rojo}) + \beta_2(\text{Dummy_Verde}) +$$

- Una Permite que las variables categóricas influyan en el modelo de manera cuantitativa.
- Los coeficientes representan la diferencia respecto a la categoría de referencia.

²Si la transformaras directamente a números (por ejemplo, rojo = 1, azul = 2, verde = 3), el modelo asumiría que hay un orden entre esos valores (como si verde fuera "mayor" que azul, y azul mayor que rojo).

■ Wrapper (Envoltorios)

- Usan el modelo para evaluar subconjuntos de variables.
- Ejemplos: Forward Selection, Backward Elimination, RFE.
- *Más precisos, pero costosos computacionalmente.*

■ Embedded

- Selección ocurre durante el entrenamiento.
- Ejemplos: Lasso (L1), Árboles de decisión, Random Forest, XGBoost.
- *Balance entre precisión y costo.*

■ Otros enfoques

- Autoencoders, PCA/ICA, métodos híbridos.
- Útiles en problemas no supervisados o de alta dimensión.

Conceptos básicos

- **Distancia de Minkowski:** es una generalización de las distancias Euclidiana y Manhattan. Se calcula la distancia de Minkowski entre los puntos P_1 y P_2 con un parámetro p utilizando la expresión:

$$d_{\text{minkowski}} = (|x_2 - x_1|^p + |y_2 - y_1|^p)^{\frac{1}{p}} \quad (1)$$

Dependiendo del valor de $\underline{p}$:

- Si $p = 1$, se obtiene la distancia Manhattan.
- Si $p = 2$, se obtiene la distancia Euclidiana.
- Si $p = \infty$, se obtiene la distancia Chebyshev.

■ Caso cuando $p \rightarrow \infty$:

- Considere $a = |x_2 - x_1|$ y $b = |y_2 - y_1|$.
- Si $p \rightarrow \infty$, entonces $a^p + b^p$ estará dominado por el mayor de los dos términos.
- Suponiendo que $a \geq b$, entonces $a^p + b^p \approx a^p$ cuando p es muy grande.

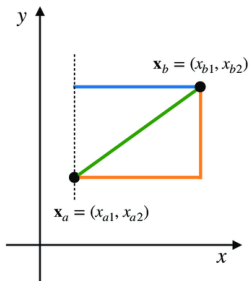
■ Límite:

$$d_\infty = \lim_{p \rightarrow \infty} (a^p + b^p)^{\frac{1}{p}} \approx \lim_{p \rightarrow \infty} (a^p)^{\frac{1}{p}} = a$$

■ Resultado:

- Cuando $p \rightarrow \infty$, $d_{\text{minkowski}}$ se convierte en $\max(a, b)$, que es la **Distancia de Chebyshev**.

Distancia de Minkowski



$p = 2$ Euclidean distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_2 = (|x_{a1} - x_{b1}|^2 + |x_{a2} - x_{b2}|^2)^{\frac{1}{2}}$$

$p = 1$ Manhattan distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_M = |x_{a1} - x_{b1}| + |x_{a2} - x_{b2}|$$

$p = \infty$ Chebyshev distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_{\infty} = \max\{|x_{a1} - x_{b1}|, |x_{a2} - x_{b2}|\}$$

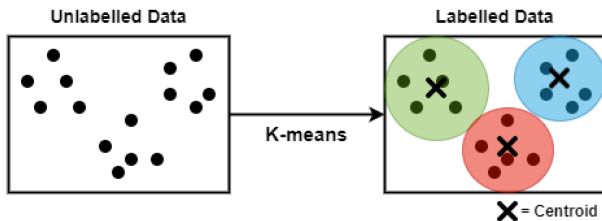
3

Cuando el parámetro p tiende a infinito, se define como el máximo de las diferencias absolutas entre las coordenadas de dos puntos.

³Fu, C., Yang, J. (2021). Granular classification for imbalanced datasets: a minkowski distance-based method. Algorithms, 14(2), 54.

K-means

K-means Clustering



- **K-means clustering** es una técnica en la que colocamos cada observación en un conjunto de datos en uno de K clusters.
- El objetivo es tener K clusters agrupando las observaciones similares entre sí.
- En la práctica, utilizamos los siguientes pasos para realizar K-means clustering:

- **Clustering** es el proceso de dividir todo el conjunto de datos en grupos basados en los patrones en los datos.
- El principal objetivo del algoritmo K-Means es minimizar la suma de las distancias entre los puntos y su respectivo centroide del cluster.

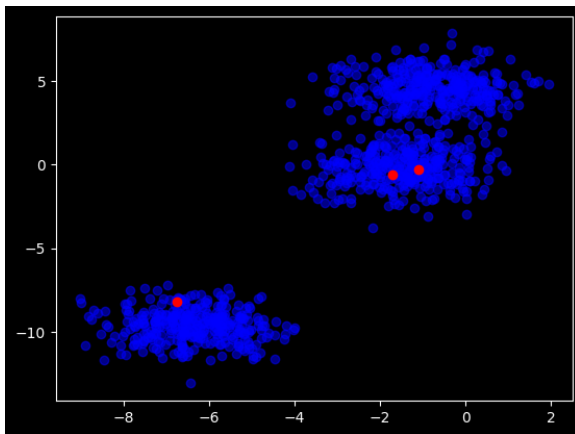
Pasos del Algoritmo K-Means:

1. Elegir el número de clusters k .
2. Seleccionar k puntos aleatorios del conjunto de datos como centroides.
3. Asignar todos los puntos al centroide del cluster más cercano.
4. Recalcular los centroides de los clusters recién formados.
5. Repetir los pasos 3 y 4 hasta que los centroides no cambien.

K-means

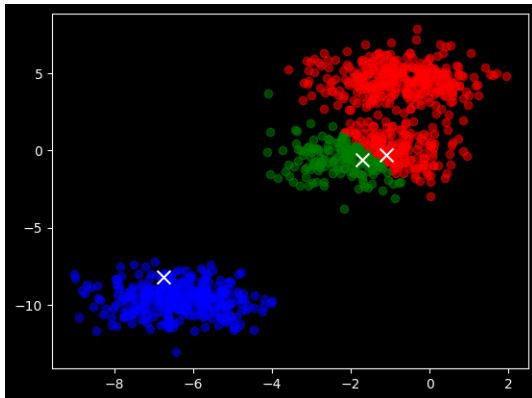
Dado un dataset sintético creado con `make_blobs` que contiene 1000 ejemplos, 2 características y clusters.

1. **Paso 1:** K igual a 3.
2. **Paso 2:** Inicializar los centroides aleatoriamente⁴.



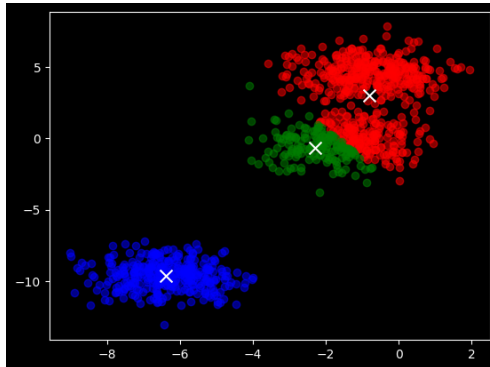
⁴Selecciona 3 ejemplos de los datos.

3. **Paso 3:** Asignar cada punto de datos al cluster más cercano.
- Se inicializa una lista de clusters vacíos y luego, para cada punto en el conjunto de datos X .
 - Se calcula la distancia euclidiana desde ese punto hasta cada uno de los centroides

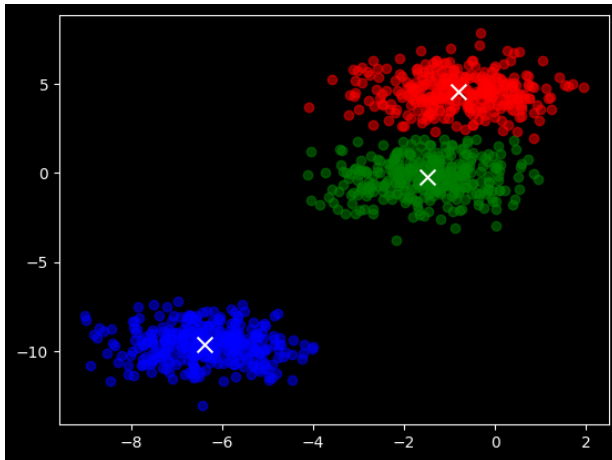


4. Paso 4: Asignar cada punto de datos al cluster más cercano.

- Se recalcula los centroides de cada cluster tomando la media de los puntos asignados a ese cluster.
- Se inicializa una matriz de centroides y se calcula la media de los puntos en ese cluster, actualizando la posición del centroide.
- Se repite hasta que los centroides se estabilizan, permitiendo que los clusters converjan a su forma final.



5. **Paso 5:** Repetir pasos 3 y 4 hasta que los centroides no cambien.



¿Qué es `kmeans.inertia_`?

- `kmeans.inertia_` es un atributo de un objeto K-Means en `scikit-learn` que representa la suma de las distancias al cuadrado del modelo de clustering K-means.
- La inercia es la suma de las distancias al cuadrado de cada punto de datos a su centroide más cercano.
- Es una medida de qué tan bien se agrupan los puntos dentro de sus respectivos clusters.

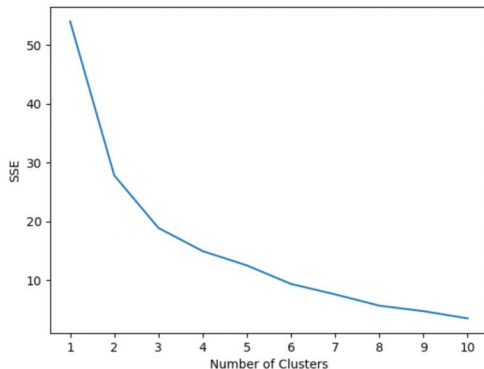
La suma de distancias al cuadrado se define matemáticamente como:

$$SSE = \sum_{i=1}^n \min_{c \in \{1, \dots, K\}} \|x_i - \mu_c\|^2$$

donde:

- n es el número de puntos de datos.
- K es el número de clusters.
- x_i es el vector de características del punto de datos i .
- μ_c es el centroide del cluster c .
- $\|x_i - \mu_c\|^2$ es la distancia al cuadrado entre el punto de datos i y el centroide c .

- **Valores Bajos de Inercia:** Indican que los puntos dentro de un cluster están cerca de su centroide.
- **Valores Altos de Inercia:** Pueden indicar que los puntos no están bien agrupados o que hay mucha dispersión dentro de los clusters.
- Al evaluar diferentes valores de K (número de clusters), los modelos con menor inercia suelen considerarse mejores, ya que implican clusters más compactos.



- El **método del codo** es una técnica común para identificar el K óptimo, buscando el punto donde la disminución de la inercia empieza a ser menos pronunciada.
- Sum of Squared Errors (SSE).

- **Problema Clásico en K-means:** La calidad del clustering en K-means depende fuertemente de la inicialización de los centroides. Una mala inicialización puede conducir a soluciones subóptimas o a la convergencia en mínimos locales.
- **K-means++**
 - Introducido para mejorar la robustez del K-means.
 - Elegir centroides iniciales que estén distanciados de manera óptima.
 - Selecciona el primer centroide de manera aleatoria.
 - Para cada punto restante, selecciona un nuevo centroide con una probabilidad proporcional al cuadrado de la distancia mínima entre ese punto y cualquier centroide ya elegido.
 - Repite hasta que se hayan seleccionado K centroides.

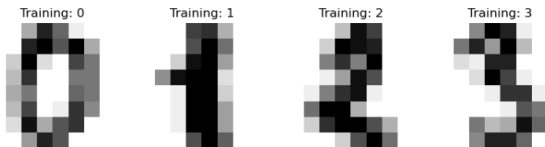
En un conjunto de datos con puntos muy dispersos, `k-means++` se enfoca en que los primeros centroides estén en diferentes partes del espacio de datos, mientras que el `k-means` puede seleccionar varios centroides muy cercanos entre sí, lo que puede llevar a un mal rendimiento.

- Después de seleccionar el primer centroide al azar, los siguientes centroides se seleccionan utilizando una probabilidad proporcional.
- Para cada punto x , se calcula la distancia $D(x)$ al centroide más cercano que ya ha sido seleccionado.
- El siguiente centroide se selecciona con probabilidad proporcional a $D(x)^2$, es decir:

$$P(x) = \frac{D(x)^2}{\sum_{x \in \text{datos}} D(x)^2}$$

- Esto significa que los puntos más lejanos de los centroides seleccionados tienen una mayor probabilidad de ser elegidos como nuevos centroides.

Ejemplo de K-means - Recognizing hand-written digits



- **Nombre:** Dígitos Escritos a Mano (Handwritten Digits)
- **Imágenes:** 1797 imágenes de 8x8 píxeles
- **Clases:** 10 (dígitos del 0 al 9)
- **Etiquetas:** Cada imagen está etiquetada con el dígito correspondiente
- **Objetivo:** Clasificar correctamente las imágenes en una de las 10 categorías
- **Desafíos:** Variabilidad en la escritura a mano y similitud entre ciertos dígitos

- Compararemos tres enfoques:
 - Una inicialización usando **k-means++**. Este método es estocástico, y ejecutaremos la inicialización 4 veces.
 - Una inicialización aleatoria. Este método también es estocástico, y ejecutaremos la inicialización 4 veces.
 - Una inicialización basada en una proyección PCA. Usaremos los componentes de la PCA para inicializar KMeans. Este método es determinista y una sola inicialización es suficiente.
- En este ejemplo, comparamos las diferentes estrategias de inicialización para K-means en términos de tiempo de ejecución y calidad de los resultados.

```
1 from sklearn.cluster import KMeans
2 from sklearn.decomposition import PCA
3 kmeans = KMeans(init="k-means++", n_clusters=n_digits,
4                 n_init=4, random_state=0)
5 bench_k_means(kmeans=kmeans, name="k-means++", data=data,
6               labels=labels)
7
8 kmeans = KMeans(init="random", n_clusters=n_digits,
9                 n_init=4, random_state=0)
10 bench_k_means(kmeans=kmeans, name="random", data=data,
11               labels=labels)
12
13 pca = PCA(n_components=n_digits).fit(data)
14 kmeans = KMeans(init=pca.components_, n_clusters=n_digits,
15                 n_init=1)
16 bench_k_means(kmeans=kmeans, name="PCA-based", data=data,
17               labels=labels)
```

*n_init: number of times the k-means algorithm is run with different centroid seeds.

Ejemplo de K-means - Recognizing hand-written digits

init	time	inertia	homo	compl	v-meas	ARI	AMI	silhouette
k-means++	0.050s	69545	0.598	0.645	0.621	0.469	0.617	0.152
random	0.064s	69735	0.681	0.723	0.701	0.574	0.698	0.170
PCA-based	0.017s	69513	0.600	0.647	0.622	0.468	0.618	0.162

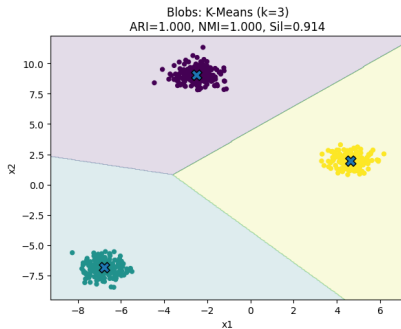
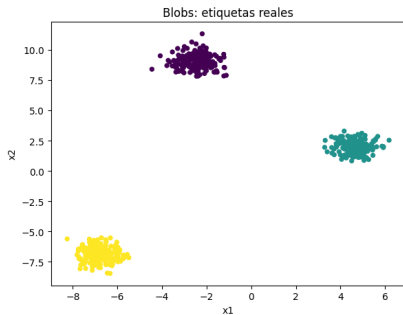
- **time:** El tiempo que tarda en ejecutarse el algoritmo.
- **inertia:** Suma de las distancias al cuadrado entre los puntos de datos y sus centroides.
- **homo (homogeneidad):** Mide qué tan homogéneos son los clusters; es decir, qué tan bien un solo cluster contiene únicamente puntos de una sola clase.
- **compl (completitud):** Mide qué tan completos son los clusters; es decir, qué tan bien todos los puntos de una clase se encuentran dentro del mismo cluster.
- **v-meas:** La media armónica de homogeneidad y completitud, una medida equilibrada entre ambas.
- **ARI (Adjusted Rand Index):** Una medida que compara la similitud entre las agrupaciones generadas por el algoritmo y una agrupación de referencia, ajustando por la posibilidad de coincidencias aleatorias.
- **AMI (Adjusted Mutual Information):** Una medida de la información mutua ajustada, que indica qué tanto se parecen las agrupaciones producidas por el algoritmo a una clasificación esperada, ajustado por la probabilidad de coincidencias aleatorias.
- **silhouette:** Un índice que mide qué tan similares son los puntos a su propio cluster en comparación con otros clusters, con valores entre -1 y 1 (cercano a 1 es mejor).

Ejemplo de K-means - Recognizing hand-written digits

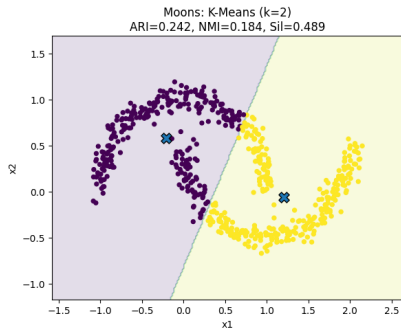
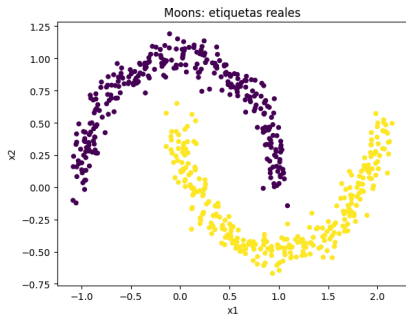
init	time	inertia	homo	compl	v-meas	ARI	AMI	silhouette
k-means++	0.050s	69545	0.598	0.645	0.621	0.469	0.617	0.152
random	0.064s	69735	0.681	0.723	0.701	0.574	0.698	0.170
PCA-based	0.017s	69513	0.600	0.647	0.622	0.468	0.618	0.162

- **v-meas:** La media armónica de homogeneidad y completitud, una medida equilibrada entre ambas.
- **ARI (Adjusted Rand Index):** Una medida que compara la similitud entre las agrupaciones generadas por el algoritmo y una agrupación de referencia, ajustando por la posibilidad de coincidencias aleatorias.
- **AMI (Adjusted Mutual Information):** Indica qué tanto se parecen las agrupaciones producidas por el algoritmo a una clasificación esperada, ajustado por la probabilidad de coincidencias aleatorias.
- **silhouette:** Un índice que mide qué tan similares son los puntos a su propio cluster en comparación con otros clusters, con valores entre -1 y 1 (cercano a 1 es mejor).

Ejemplo de K-means

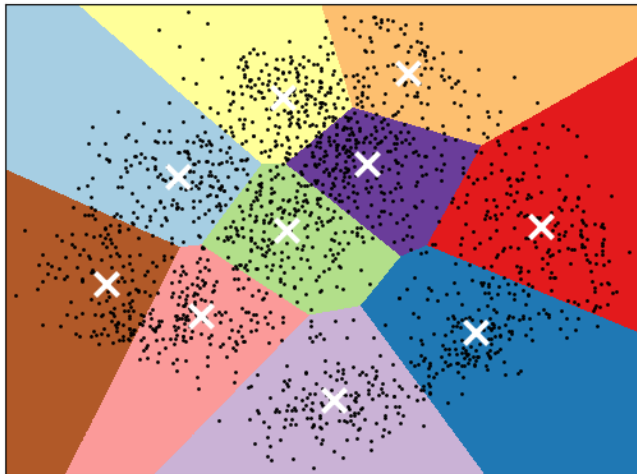


Ejemplo de K-means



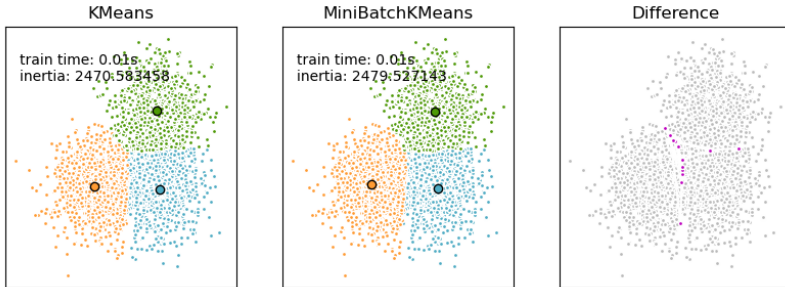
Ejemplo de K-means - Recognizing hand-written digits

K-means clustering on the digits dataset (PCA-reduced data)
Centroids are marked with white cross



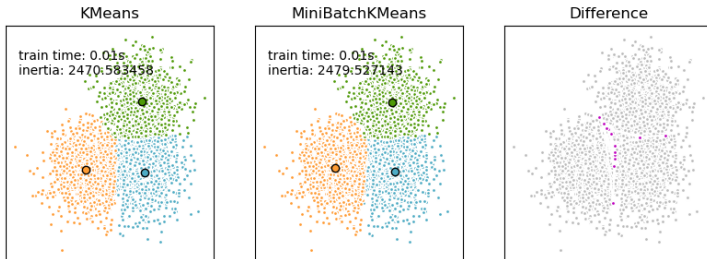
- K-means estándar puede ser computacionalmente costoso y lento en datasets grandes debido a la necesidad de recalcular los centroides con cada iteración sobre todos los datos.
- **Método:**
 - En lugar de usar el conjunto de datos completo en cada iteración, MiniBatchKMeans toma mini-batches de datos aleatoriamente.
 - Actualiza los centroides basándose solo en el mini-lote actual, lo que reduce drásticamente el tiempo de cálculo.

K-means vs. MiniBatchMeans



- **MiniBatchKMeans** es una variante del algoritmo K-Means que utiliza mini-batches para reducir el tiempo de cálculo.
- Aunque reduce el tiempo de cálculo, aún intenta optimizar la misma función objetivo que el K-Means estándar.
- **Mini-batches:** Son subconjuntos de los datos de entrada, muestreados aleatoriamente en cada iteración de entrenamiento.
- **Ventaja:** Los mini-batches reducen drásticamente la cantidad de cálculo necesario para converger a una solución local.
- **Criterio de parada:** Los pasos se repiten hasta que se alcanza la convergencia o un número predeterminado de iteraciones.

K-means vs. MiniBatchMeans (2)



- Por lo general, MiniBatchKMeans produce resultados con menor desempeño que el algoritmo estándar.
- Se extraen muestras aleatorias del dataset (mini-batch).
- Estas muestras se asignan al centroide más cercano.
- Los centroides se actualizan, pero a diferencia de K-Means, esto se hace por muestra.
- Para cada muestra en el mini-batch, el centroide asignado se actualiza tomando el promedio acumulativo de la muestra y todas las muestras anteriores asignadas a ese centroide. Esto disminuye la tasa de cambio de un centroide a lo largo del tiempo.
- **Criterio de parada:** Los pasos se repiten hasta que se alcanza la convergencia o un número predeterminado de iteraciones.

La actualización de centroides en MiniBatchMeans se realiza utilizando la siguiente fórmula:

$$C_{t+1} = C_t + \frac{1}{N_i} (x_i - C_t)$$

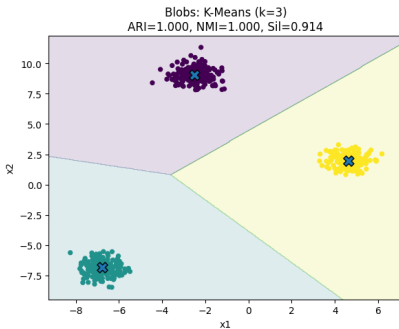
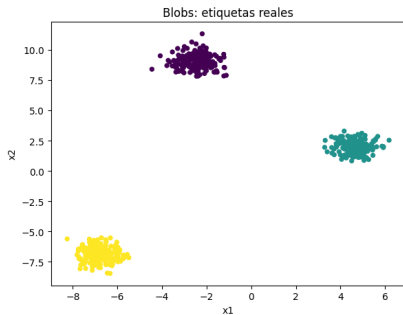
- x_i : Es el punto de datos del mini-batch actual que ha sido asignado al centroide. Representa un vector de características del conjunto de datos.
- C_t : Es la posición actual del centroide en la iteración t .
- C_{t+1} : Es la nueva posición del centroide después de la actualización en la iteración $t + 1$.
- N_i : Es el número de puntos que han sido asignados al centroide C hasta esa iteración.

La actualización de centroides en MiniBatchMeans se realiza utilizando la siguiente fórmula:

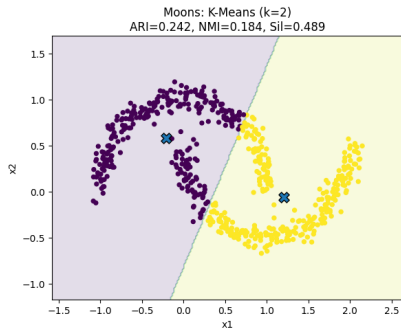
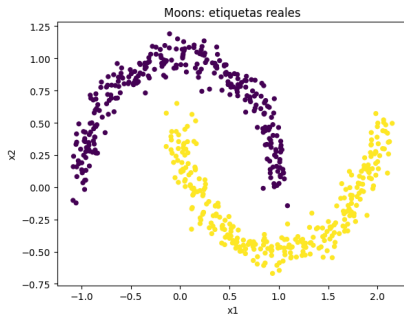
$$C_{t+1} = C_t + \frac{1}{N_i}(x_i - C_t)$$

- x_i : Es el punto de datos del mini-batch actual que ha sido asignado al centroide. Representa un vector de características del conjunto de datos.
- C_t : Es la posición actual del centroide en la iteración t .
- C_{t+1} : Es la nueva posición del centroide después de la actualización en la iteración $t + 1$.
- N_i : Es el número de puntos que han sido asignados al centroide C hasta esa iteración.
- Se ajusta el centroide C_t al considerar la información del punto x_i en el mini-batch, moviéndolo ligeramente hacia x_i para que el centroide se acerque más a los puntos asignados a él.
- Este ajuste es incremental, lo que significa que el cambio en el centroide se vuelve más pequeño a medida que se procesan más puntos, estabilizando la posición del centroide con el tiempo.

Ejemplo de K-means



Ejemplo de K-means



Hierarchical Clustering

Clustering Aglomerativo (Bottom-Up)

- **Problema con K-Means:** requiere pre-especificar el número de clusters K .
 - **Alternativa:** el clustering jerárquico no requiere fijar K de antemano.
 - Además, genera una representación en forma de árbol (**dendrograma**) de las observaciones.
-
- Método más común de clustering jerárquico.
 - **Principio de funcionamiento:** comenzar con cada observación como un cluster individual.
 - Paso a paso:
 1. Calcular distancias entre todas las observaciones.
 2. Fusionar los dos clusters más cercanos.
 3. Repetir hasta que todas las observaciones estén en un único cluster.
 - El resultado se representa como un **dendrograma**.

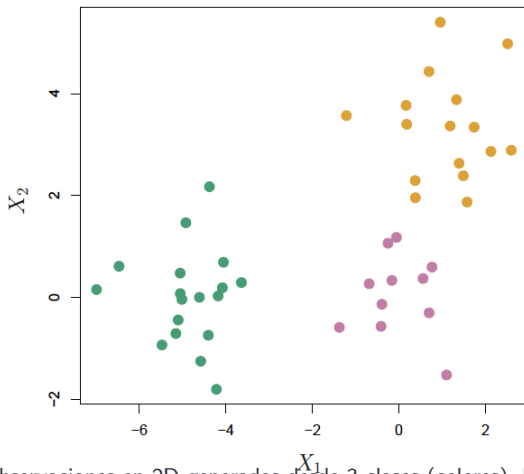


Figura 1: 45 observaciones en 2D generadas desde 3 clases (colores). Los labels reales se omiten para aplicar clustering jerárquico.

Interpretando un Dendrograma

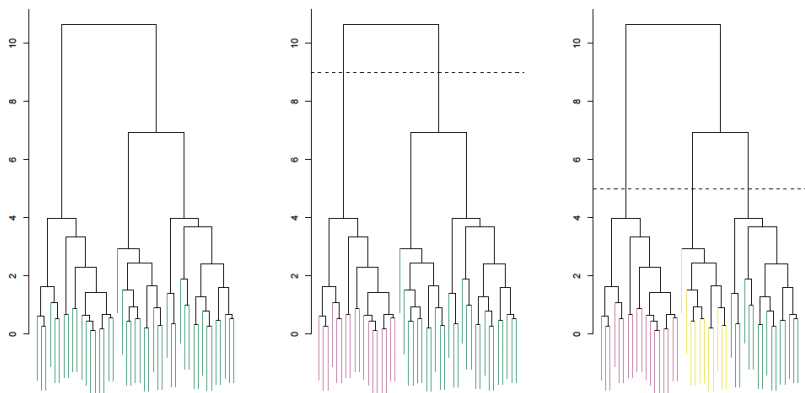


Figura 2: Dendrograma con complete linkage y distancia euclidiana. Corte a altura 9 produce 2 clusters; corte a altura 5 produce 3 clusters (colores solo ilustrativos).

Interpretando un Dendrograma

- Cada **hoja** representa una observación.
 - A medida que subimos en el árbol, las observaciones y clusters se fusionan.
 - **Altura de la fusión:** mide la disimilitud entre los grupos que se unen.
 - Fusión baja → observaciones muy similares.
 - Fusión alta → observaciones muy diferentes.
 - **Importante:** la proximidad horizontal NO indica similitud; lo relevante es la **altura de la unión**.
-
- Un **corte horizontal** en el dendrograma determina el número de clusters.
 - Ejemplo:
 - Corte a altura 9 → 2 clusters.
 - Corte a altura 5 → 3 clusters.
 - El **nivel de corte** juega un rol similar al K en K-Means.

Algoritmo de Clustering Jerárquico (Aglomerativo)

1. Comenzar con n observaciones y una medida de disimilitud (ej. distancia euclidiana) de todas las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes por pares. Tratar cada observación como su propio cluster.

2. Para $i = n, n-1, \dots, 2$:
 - a) Identificar el par de clusters con menor disimilitud (más similares). Fusionarlos $\rightarrow$ la disimilitud indica la **altura en el dendrograma**.
 - b) Calcular las nuevas disimilitudes entre el nuevo cluster y los $i-1$ restantes.

Idea clave: Cada paso une los clusters más parecidos.

Resultado: un dendrograma que muestra cómo se fusionan hasta formar un solo cluster.

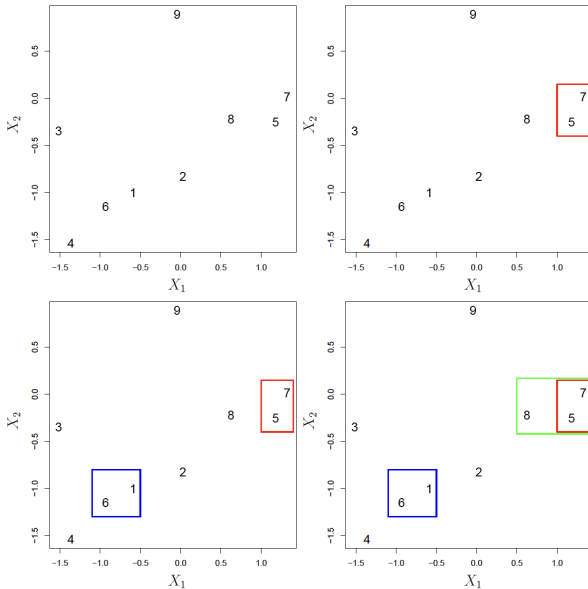
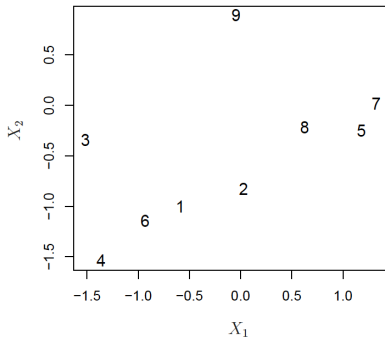
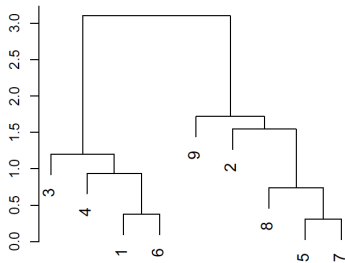


Figura 3: Primeros pasos del clustering jerárquico con complete linkage y distancia euclidiana. Arriba-Izq: 9 clusters iniciales. Arriba-Der: fusión de {5} y {7}. Abajo-Izq: fusión de {6} y {1}. Abajo-Der: fusión de {8} con el cluster {5,7}.

Corte del Dendrograma



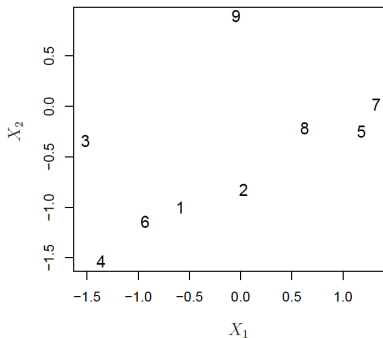
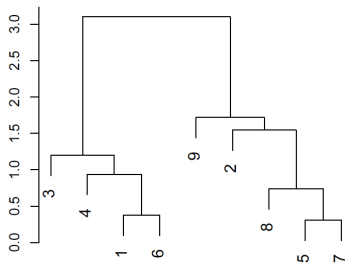
Ventajas

- No requiere pre-especificar K .
- Da visión jerárquica completa de los datos.
- Dendrograma facilita interpretación visual.

Desventajas

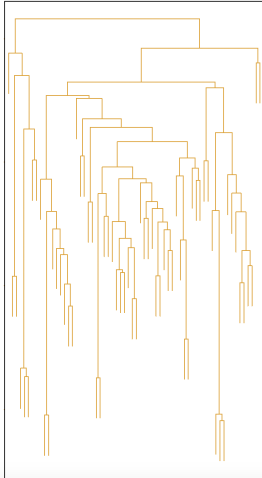
- Computacionalmente costoso ($O(n^2)$).
- Sensible a la métrica de distancia y al criterio de enlace.

Criterios de Enlace en Clustering Jerárquico

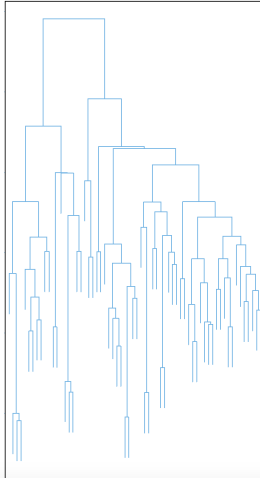


Linkage	Descripción
Complete	Máxima disimilitud entre clusters. Se calculan todas las distancias entre observaciones de A y B , y se toma la <u>mayor</u> .
Single	Mínima disimilitud entre clusters. Se calculan todas las distancias entre observaciones de A y B , y se toma la <u>menor</u> . Puede generar “cadenas” largas de uniones.
Average	Media de las disimilitudes entre observaciones de A y B . Se registra el <u>promedio</u> .
Centroid	Distancia entre los centroides de los clusters A y B . Puede producir inversiones indeseadas en el dendrograma.

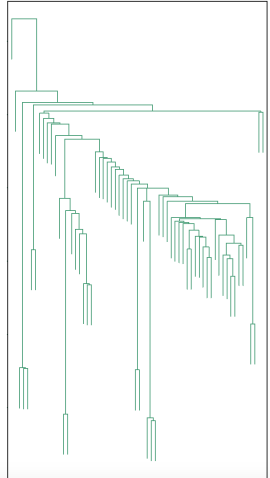
Average Linkage



Complete Linkage



Single Linkage



K-Nearest Neighbors

K-Nearest Neighbors (KNN) algorithm

- Algoritmo KNN es uno de los algoritmos clásicos de aprendizaje automático supervisado.
- KNN es capaz de clasificar tanto en binario como en multiclase.
- Algoritmo de clasificación no paramétrico. No paramétrico por naturaleza, KNN también se puede utilizar como algoritmo de regresión.

- KNN es un algoritmo no paramétrico. Por lo tanto, el entrenamiento de un clasificador KNN no requiere pasar por el enfoque más tradicional de iterar sobre los datos de entrenamiento durante varias épocas para optimizar un conjunto de parámetros.
- El entrenamiento de KNN implica ajustar (o guardar) todas las instancias de datos de entrenamiento en la memoria del equipo al mismo tiempo.
- Etapa de inferencia, el modelo compara estos nuevos datos con las instancias de datos de entrenamiento existentes.

- **Cálculo de la distancia:** El modelo KNN calcula la distancia⁵ entre el nuevo punto de datos y todos los puntos de datos en el conjunto de datos de entrenamiento.
- **Selección de vecinos más cercanos:** El algoritmo selecciona un número k de puntos de datos de entrenamiento más cercanos al nuevo punto de datos.
- **Asignación de la clase:** El algoritmo asigna al nuevo punto de datos la clase que aparece con mayor frecuencia entre estos k vecinos.

⁵Distancia euclidiana

Introduction to K-Nearest Neighbors

- **Objetivo de KNN:** Predecir respuestas cualitativas (clasificación) basadas en la proximidad a puntos de entrenamiento conocidos.
- El clasificador KNN intenta aproximar la distribución condicional de Y dado X sin asumir un modelo paramétrico.
- **Ventajas:**
 - No requiere suposiciones sobre la distribución de los datos.
 - Es simple y fácil de entender.
- **Desventajas:**
 - Sensible al valor de K y a la métrica de distancia utilizada.
 - Computacionalmente costoso para grandes datasets.

Definición de KNN:

- Dado un entero positivo K y una observación de prueba x_0 , el clasificador KNN identifica los K puntos más cercanos en los datos de entrenamiento.
- Estos puntos más cercanos se representan como $\mathcal{N}_0$.

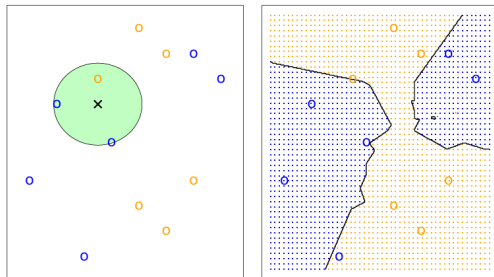
Definición de KNN:

- Dado un entero positivo K y una observación de prueba x_0 , el clasificador KNN identifica los K puntos más cercanos en los datos de entrenamiento.
- Estos puntos más cercanos se representan como $\mathcal{N}_0$.
- La probabilidad condicional estimada para la clase j se calcula como la fracción de puntos en $\mathcal{N}_0$ cuyas respuestas corresponden a la clase j :

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j).$$

- Aquí, $\mathcal{N}_0$ representa el conjunto de los K puntos más cercanos a la observación de prueba x_0 , e $I(y_i = j)$ es una función indicadora⁶ que es 1 si $y_i = j$ y 0 en caso contrario.
- **Clasificación:** KNN clasifica la observación de prueba x_0 a la clase con la mayor probabilidad estimada, de acuerdo con la ecuación anterior.

⁶Métricas de distancia



- El enfoque KNN, utilizando $K=3$, se ilustra en una situación simple con seis observaciones azules y seis observaciones naranjas.
- **Izquierda:** Se muestra una observación de prueba, donde se desea predecir una etiqueta de clase. Se identifican los tres puntos más cercanos a la observación de prueba, y se predice que la observación de prueba pertenece a la clase que ocurre con mayor frecuencia (azul).
- **Derecha:** Se muestra la frontera de decisión del KNN para este ejemplo en negro.

- `K-Neighbors Classifier` maneja un parámetro para identificar la métrica de distancia, llamado *metric*, el cual por defecto tiene como valor la distancia de **Minkowski**.
- `K-NeighborsClassifier` maneja un parámetro **p** para definir la **potencia** aplicada a la métrica distancia de Minkowski, siendo su valor por defecto $p = 2$.
- ¿Qué valor debíamos configurar para p ?

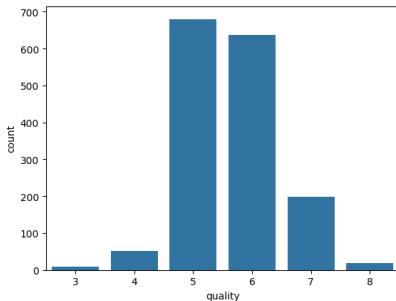
- UCI Red Wine Dataset: Contiene 1,599 muestras de vino tinto.
- El objetivo es predecir la **calidad del vino**, que es una variable ordinal en una escala de 0 a 10.
- **Variables del Dataset:**
 - **Fixed acidity:** Acidez fija (no volátil).
 - **Volatile acidity:** Acidez volátil (afecta al aroma).
 - **Citric acid:** Ácido cítrico (contribuye a la frescura).
 - **Residual sugar:** Azúcar residual (después de la fermentación).
 - **Chlorides:** Nivel de sal.
 - **Free sulfur dioxide:** Sulfuro de dióxido libre (conservante).
 - **Total sulfur dioxide:** Dióxido de azufre total.
 - **Density:** Densidad del vino.
 - **pH:** Mide la acidez.
 - **Sulphates:** Sulfatos (conservante).
 - **Alcohol:** Contenido de alcohol en el vino.

Ejemplo de KNN Classifier

- Para prevenir inestabilidades en el proceso de entrenamiento, estandarizamos el tensor de entrada X y el tensor de salida Y que contiene todos los pares de imágenes de entrada-salida de entrenamiento:

$$\bar{X} = \frac{X - \mu_X}{\sigma_X}, \quad \bar{Y} = \frac{Y - \mu_Y}{\sigma_Y} \quad (2)$$

- Ninguna muestra de vino recibe una calificación superior a 8 ni inferior a 3⁷⁸. Estos casos pueden considerarse una situación hipotéticamente ideal, o probablemente esos datos sufren un sampling bias.



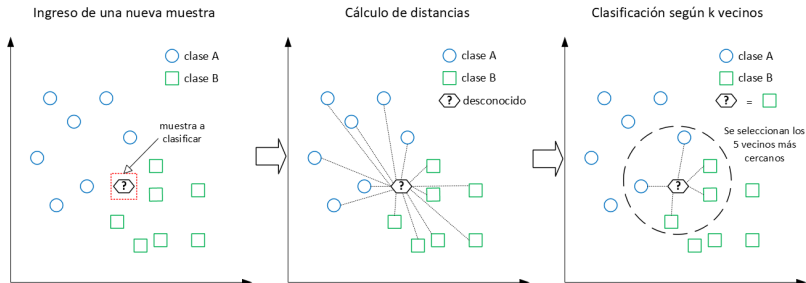
⁷StratifiedKFold

⁸AUC-ROC ó vecinos más cercanos con más influencia usando la inversa de la distancia

Ejemplo de KNN Classifier

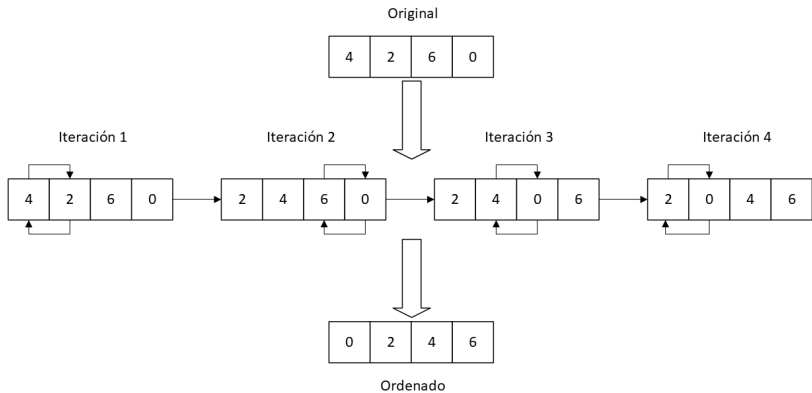
- **Paso 1:** Guardar el dataset en la memoria (**Equivilante al entrenamiento**). Dividir el dataser en training y test set.
- **Paso 2:** Inferencia en el algoritmo KNN, el proceso de cálculo de la distancia es un proceso iterativo en el que calculamos la distancia euclidiana de un punto de datos en los datos de prueba con respecto a cada punto de datos en los datos de entrenamiento.
- **Paso 3:** $d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$.
- Selecciona los k puntos de datos de entrenamiento más cercanos al punto de datos de prueba basándose en las distancias calculadas. **La etiqueta con la mayor frecuencia de aparición se asigna como clase objetivo al punto de datos de prueba.**

Ejemplo de KNN Classifier

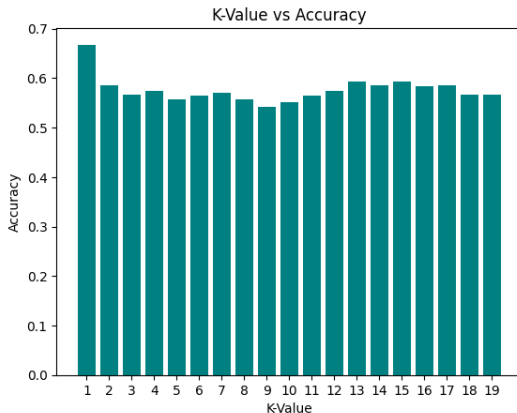


Para visualizar problemas con más de 2 o 3 dimensiones, se recomienda usar PCA

Ejemplo de KNN Classifier



Impacto del valor de k en KNN



Como podemos ver, el valor de $k = 1$ tiene el mayor accuracy. Pero, para $k = 1$, el modelo será muy sensible a los *outliers*. Por lo tanto, optaremos por $k = 12$, que tiene el segundo mejor accuracy (0.59) .

■ Valor de k bajo:

- Un k bajo (e.g., 1 o 2) hace que el modelo sea muy sensible a los **outliers**.
- Los **outliers** son instancias extremas que no siguen las tendencias generales del conjunto de datos.
- Como resultado, las predicciones del modelo se vuelven inestables.

■ Valor de k alto:

- A medida que aumenta k , se observa un aumento inicial en la estabilidad del algoritmo.
- Con más vecinos, el efecto de los outliers disminuye debido al voto mayoritario.
- Sin embargo, al seguir aumentando k , la estabilidad comienza a disminuir y la precisión del modelo se deteriora.

- KNN es costoso computacionalmente.
- Conforme la dimensionalidad y la escala del dataset aumenta, el tamaño del modelo aumenta, lo que impacta el rendimiento y complejidad del modelo.
- KNN es sensible a los outliers por lo que pequeños cambios en los datos de entrada (noise) pueden afectar el rendimiento del modelo.

- Algoritmo de **aprendizaje supervisado** para problemas de **regresión**.
- Predice el valor de una muestra basándose en los valores de sus k vecinos más cercanos.

Pasos para KNN Regressor

- Se calcula la distancia entre el punto de interés y los puntos del conjunto de entrenamiento.
- Se seleccionan los k puntos más cercanos.
- El **valor predicho** es la **media** (o una combinación) de los valores de esos k vecinos más cercanos.
 - Un k pequeño puede captar más **ruido**.
 - Un k grande puede hacer la predicción demasiado **general**.
- La distancia **euclidiana** es la más común, pero también se pueden usar otras como **Manhattan** o **Minkowski**.

Técnicas de evaluación de algoritmos de clasificación

Ground-truth

Información pre-definida que representa el resultado esperado para la resolución de cierto problema.

- Esta información se utiliza como base de comparación cuando se evalúan algoritmos para la resolución del mismo problema.
- En imágenes y video, cuando se mide la habilidad de cierto algoritmo para enfrentar cierto problema, en general, se utilizan métricas que puedan mostrar:
 - **Un valor definido** (e.g., error de localización 2D, intervalo de tiempo entre la aparición del objeto en el ground-truth y su detección en el sistema, ...). En este caso, se pueden obtener estadísticos de estos valores (media, desviación estándar, mínimo y máximo, ...) o ver su evolución en el tiempo.

- ... en general, se utilizan métricas que puedan mostrar:
 - **Un número de éxitos o fallas del algoritmo** (proviene de clasificación binaria). En este caso, se usan las siguientes definiciones:
 - **Verdadero Positivo (TP)**: El algoritmo detecta una situación real (existe en la ground-truth y en los resultados del algoritmo).
 - **Falso Negativo (FN)**: No se detectó una situación real (existe sólo en la ground-truth).
 - **Falso Positivo (FP)**: El algoritmo detecta una situación irreal (existe sólo en los resultados del algoritmo).
 - **Verdadero Negativo (TN)**: Elemento que no aparece ni en la ground-truth ni en los resultados del algoritmo.

- TP, FP, TN, FN también pueden dar más información para caracterizar a los algoritmos:

- **Precision** = $\frac{TP}{TP + FP}$, tasa de aciertos, contra aciertos y errores del algoritmo⁹.
- **Sensitivity** = $\frac{TP}{TP + FN}$, tasa de aciertos, contra todo lo que se debía detectar¹⁰.
- **Specificity** = $\frac{TN}{FP + TN}$, tasa de elementos correctamente no detectados, contra todo lo que no debía generar respuesta.
- **F-score** = $2 * \frac{\text{Precision} * \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$, media armónica entre Precision y Sensitivity.
- **Acc:** = $\frac{TN + TP}{FP + FN + TN + TP}$

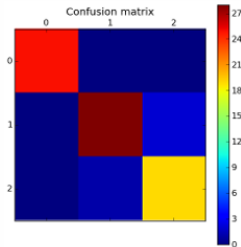
⁹El objetivo es medir la capacidad del modelo para no identificar erróneamente una señal negativa como positiva.

¹⁰Este criterio mide la capacidad del modelo para encontrar todas las señales positivas en el conjunto de datos.

Evaluación: Matriz de Confusión (Confusion Matrix)

- Es una herramienta de visualización típicamente usada en aprendizaje supervisado (muestras de entrenamiento etiquetadas).
- Cada columna de la matriz representa las instancias de clases predichas (e.g., la salida de un algoritmo).
- Cada fila representa las verdaderas instancias de la clase (e.g., etiquetas de las muestras, conocidas a priori).
- También, se encuentra la misma representación matricial transpuesta (en este caso, normalmente se aclara).

		prediction outcome		total
		p	n	
actual value	p'	True Positive	False Negative	P'
	n'	False Positive	True Negative	N'
total		P	N	



- Es fácil ver si el sistema se confunde entre dos o más clases.
- Cuando un conjunto de datos es desbalanceado (número de muestras en diferentes clases varía mucho) la tasa de error del clasificador no es representativa del verdadero rendimiento del clasificador.
- e.g. clase A: 990 muestras - clase B: 10 muestras: **¿Qué pasa con la tasa de error si acierto en todas con A y en ninguna con B?**.acierto

- **Receiver Operating Characteristic (ROC)**, o curva ROC, es una gráfica que ilustra el rendimiento de un clasificador binario, ante la variación de su umbral.
- Se crea graficando la **TPR** (true positive rate) vs la **FPR** (false positive rate), en distintos umbrales.

Definiciones Clave

- **TPR (True Positive Rate):**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \text{Sensibilidad}$$

- **FPR (False Positive Rate):**

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Especificidad}$$

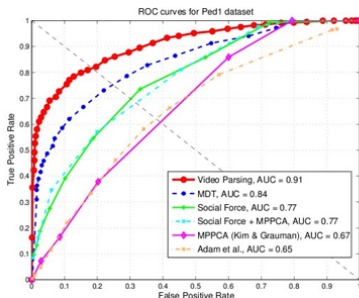
Definiciones Clave

- TPR (True Positive Rate):

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \text{Sensibilidad}$$

- FPR (False Positive Rate):

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Especificidad}$$



Búsqueda en Malla (Grid Search)

- **Grid Search** es una técnica de optimización utilizada para encontrar la combinación óptima de hiperparámetros en un modelo de ML.
- Se define una malla de posibles valores para cada hiperparámetro.
- El modelo se entrena y evalúa exhaustivamente con cada combinación posible de hiperparámetros en la malla.

Pasos del Grid Search

1. **Definir los hiperparámetros:** Seleccionar los hiperparámetros del modelo que se desean ajustar.
 2. **Establecer el rango de valores:** Especificar los valores posibles para cada hiperparámetro.
 3. Entrenar el modelo con cada combinación de hiperparámetros y evaluarlo utilizando un método como la validación cruzada.
 4. **Seleccionar la mejor combinación:** Escoger la combinación de hiperparámetros que proporcione el mejor rendimiento según la métrica de evaluación seleccionada.
- Puede ser computacionalmente costoso si hay muchos hiperparámetros o un gran número de valores posibles para cada uno.
 - *Random Search* o técnicas más avanzadas como *Bayesian Optimization*.

- Definir un modelo de machine learning. Este modelo será entrenado múltiples veces con diferentes combinaciones de hiperparámetros.
- GridSearchCV entrena el modelo en todas las combinaciones de los hiperparámetros que definiste, evaluando cada combinación en la validación cruzada y almacenando los resultados.
- Selección del mejor modelo: Después de probar todas las combinaciones, selecciona la que dio los mejores resultados según la métrica de evaluación definida (como accuracy, F-score, RMSE, entre otras).



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Aprendizaje automático - Sesión 4

Feature Selection & K-nearest Neighbors & K-means Algorithms

Doctorado en Inteligencia Artificial

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile